Sparse hard oblique forests for jet classification

28 October 2026

Abstract

We study compact classifiers built from shallow oblique trees with linear leaves. Tree Alternating Optimization updates their cuts and leaf functions against a common forest loss. To reduce the number of coefficients, we select support with an L1 penalty, freeze it, and refit without shrinkage (Select–Freeze–Refit). In a jet-classification application, three depth-four trees with 418 nonzero coefficients achieve 56.20% signal efficiency at 1% background acceptance, compared with 55.48% for a 100-tree XGBoost reference with 3,099 scalar parameters. The results motivate sparse oblique forests as candidates for resource-constrained inference.

1 Introduction

Fast classification under hardware resource constraints motivates models that combine predictive accuracy with a small arithmetic workload. An axis-aligned decision tree tests one feature at each node; an oblique tree tests a linear combination. Oblique cuts can describe correlated decision boundaries within shallow trees. Linear leaf functions add first-order variation inside each region. Their coefficients also introduce a cost: a general affine function requires several multiplications and additions.

We investigate this tradeoff using additive forests of hard oblique trees. Tree Alternating Optimization (TAO) revisits the cuts of an existing tree, rather than fixing each cut in a single top-down construction [1]. Sparse oblique trees have also been used to analyze learned feature representations [2]. Here we optimize a shared classification loss for all forest members and study L1 regularization followed by fixed-support refitting. This separates coefficient selection from shrinkage, following the motivation of post-selection methods such as relaxed Lasso [3].

Our application distinguishes quark/gluon jets from W, Z and top jets using 16 high-level input features. The intended implementation context is FPGA inference for the CMS Phase-II Level-1 trigger. We compare floating-point classification performance, coefficient counts and arithmetic bounds, providing a basis for subsequent quantization and synthesis studies.

2 Oblique forests

2.1 Cuts, paths and leaf functions

An event is represented by a feature vector $x \in \mathbb{R}^F$ and a class label $y \in \{0, \dots, K-1\}$. A forest contains T binary trees of depth D, where depth counts decisions on a full root-to-leaf path. We label a node by its path p from the root: $p = \varnothing$ denotes the root, L its left child, R its right child, and RL the left child of R. Internal paths have length $|p| < D$; leaf paths have length D. Appending L or R identifies the two children. This notation describes the tree independently of storage.

At internal node p of tree $t \in \{1, \ldots, T\}$, an affine function

$$h_{tp}(x) = w_{tp}^{\mathsf{T}} x + b_{tp}, \qquad p \longrightarrow \begin{cases} pR, & h_{tp}(x) > 0, \\ pL, & h_{tp}(x) \leq 0, \end{cases} \tag{1}$$

determines the next branch. Here $w_{tp} \in \mathbb{R}^F$ is the cut's feature weight vector and $b_{tp} \in \mathbb{R}$ its bias. Routing is hard: each event follows exactly one branch. A cut is axis-aligned when only one component of w_{tp} is nonzero, and oblique when it combines several features.

A complete depth-D tree has $I = 2^D - 1$ internal nodes and $L_D = 2^D$ leaves. Its effective topology can be smaller. If $w_{tp} = 0$, the bias determines a constant route, and the other child and its descendants are unreachable for *every* input. For example, $w_{tp} = 0, b_{tp} = 0$ always chooses the left child by Eq. (1). Such branches can be removed without changing the function. A branch merely unused by a finite data sample is not necessarily unreachable.

Let $\ell_t(x)$ be the leaf path reached in tree t. With class $K - 1$ as a reference of zero logit, a linear leaf stores $K - 1$ affine functions

$$f_{t\ell k}(x) = a_{t\ell k}^{\mathsf{T}} x + c_{t\ell k}, \qquad k = 0, \ldots, K - 2, \tag{2}$$

where $a_{t\ell k} \in \mathbb{R}^F$ and $c_{t\ell k} \in \mathbb{R}$ are the leaf weights and biases. A constant leaf restricts $a_{t\ell k} = 0$. The forest logits and probabilities are

$$z_k(x) = \sum_{t=1}^{T} f_{t,\ell_t(x),k}(x), \quad z_{K-1}(x) = 0, \qquad P(y = k \mid x) = \frac{\exp z_k(x)}{\sum_{q=0}^{K-1} \exp z_q(x)}. \tag{3}$$

Thus tree logits are added before applying softmax. Linear leaves allow first-order dependence within a region, but do not enforce continuity across its boundaries. The forest is affine on each intersection of its trees' leaf regions; adding trees refines this partition.

For the binary application, class 0 is background and class 1 is signal. The signal margin and probability are

$$m(x) = -z_0(x), \qquad P(y = 1 \mid x) = \frac{1}{1 + \exp[-m(x)]}. \tag{4}$$

A larger margin therefore corresponds to a more signal-like event.

2.2 Parameter and arithmetic counts

Counting scalar weights and biases gives

$$P_{\text{dense}} = T\left[(2^D - 1)(F + 1) + 2^D B(K - 1)\right], \qquad B = \begin{cases} F + 1, & \text{linear leaves}, \\ 1, & \text{constant leaves}. \end{cases} \tag{5}$$

This is the allocated coefficient count for a complete topology. We separately report P_{nz}, the number of coefficients exactly different from zero, and P_{reach}, the nonzero count after removing algebraically unreachable branches. These counts include biases and exclude feature indices, connectivity and preprocessing metadata.

For $F = 16, K = 2, D = 4$, one linear-leaf tree has $15 \cdot 17 + 16 \cdot 17 = 527$ coefficient slots, while three have 1,581. A single dense tree may therefore have a similar *nonzero* count to a more strongly regularized forest of three trees, despite its smaller allocated count.

A dense binary forest with linear leaves needs at most $TF(D + 1)$ feature multiplications and TD comparisons along a selected path. Bias additions are not multiplications. Evaluating all internal functions and leaves instead uses $TF(2^{D+1} - 1)$ multiplications. Sparse functions reduce these counts according to their retained feature weights. They are arithmetic bounds; latency also depends on parallelism, multiplier reuse, precision and memory.

Implementation and reproducibility

Storage and inference. `HardTree` and `HardTAOClassifier` in `src/tao/hard.py` implement Eqs. (1)–(3). Internal nodes use heap indices $j = 0, \dots, I-1$, with children $2j+1, 2j+2$; leaf-array indices subtract I from the heap index. The arrays are `split_w`, `split_b` and `leaf_parameters`. The methods `routes`, `leaf_indices`, `predict_subtree`, `decision_function` and `predict_proba` implement traversal and logits. Input and logit arithmetic is binary64; public probabilities are cast to float32. The native ROC uses those probabilities, including any ties. Portable margin inference remains binary64. The current portable schema is binary, although the fitting code supports multiclass logits.
`sparsity_profile` in `src/tao/sparsity.py` computes exact-zero and reachable counts. The existing NumPy `dense` traversal may compute a node function over an entire batch before selecting reaching rows; its runtime does not represent a generated sparse path kernel.

3 Tree Alternating Optimization

3.1 A common forest objective

For training events (x_i, y_i), $i = 1, \dots, N$, let $v_i \geq 0$ be event weights and $W = \sum_i v_i > 0$. All results below use $v_i = 1$. We minimize

$$J(\theta) = \frac{1}{W} \sum_{i=1}^{N} v_i \, \mathrm{CE}(y_i, z(x_i)) + \lambda_s \sum_{t,p} \left(\|w_{tp}\|_1 + |b_{tp}| \right) + \lambda_\ell \sum_{t,\ell,k} \|a_{t\ell k}\|_1 , \qquad \mathrm{CE}(y, z) = \log \sum_{k=0}^{K-1} e^{z_k} - z_y. \tag{6}$$

Here θ denotes all cut and leaf parameters, $\lambda_s \geq 0$ penalizes cuts and $\lambda_\ell \geq 0$ penalizes leaf feature weights. Leaf biases are unpenalized; cut biases are penalized. For constant leaves, λ_ℓ has no effect. Every subproblem retains the full training-weight denominator W, so a penalty has the same normalization at different depths and node occupancies.

TAO alternates optimization blocks while other parameters are fixed [1]. An *epoch* visits every tree, updates its leaves and then its internal nodes from the bottom up, and finally refits all forest leaves jointly. Other trees contribute fixed logit offsets during each individual-tree update. Their offsets are refreshed before updating the next tree. Earlier trees remain trainable; the procedure does not freeze them as later trees are added.

3.2 Leaf optimization

Fix every cut, so the leaf reached by each training event is fixed. For leaf ℓ of tree t, define its event set $\mathcal{I}_{t\ell} = \{i : \ell_t(x_i) = \ell\}$. Introduce the leaf basis $\phi(x) = (x_1, \dots, x_F, 1)^\mathsf{T} \in \mathbb{R}^B$ for linear leaves and $\phi(x) = 1$ for constant leaves. Let $\beta_{t\ell k}$ collect the coefficients of that basis: it is $(a_{t\ell k}, c_{t\ell k})$ for a linear leaf and just $c_{t\ell k}$ for a constant leaf. For $i \in \mathcal{I}_{t\ell}$,

$$z_{ik} = o_{ik} + \phi(x_i)^\mathsf{T} \beta_{t\ell k}, \qquad o_{ik} = \sum_{s \neq t} f_{s,\ell_s(x_i),k}(x_i), \qquad k < K-1, \tag{7}$$

with $z_{i,K-1} = o_{i,K-1} = 0$. Thus the other trees enter as a fixed offset, not as an additional target to be fitted.

Terms in Eq. (6) from other leaves and all cuts are constant in this update. Removing them leaves the subproblem

$$J_{t\ell}(\beta) = \frac{1}{W} \sum_{i \in \mathcal{I}_{t\ell}} v_i \, \mathrm{CE}(y_i, z_i(\beta)) + \lambda_\ell \sum_{b=1}^{B} \sum_{k=0}^{K-2} \rho_b |\beta_{t\ell k,b}|, \tag{8}$$

where $\rho = (1, \dots, 1, 0)$ for a linear leaf and $\rho = 0$ for a constant leaf. In particular, the bias is unpenalized and the denominator remains W, not the weight sum in $\mathcal{I}_{t\ell}$.

Write $\pi_{ik} = \exp(z_{ik}) / \sum_{j=0}^{K-1} \exp(z_{ij})$ for the current class probabilities, and let $\mathbf{1}[A]$ equal 1 when statement A holds and 0 otherwise. Differentiating the cross-entropy gives

$$\frac{\partial \operatorname{CE}(y_i, z_i)}{\partial z_{ik}} = \pi_{ik} - \mathbf{1}[y_i = k], \qquad \frac{\partial L_{t\ell}}{\partial \beta_{t\ell k,b}} = \frac{1}{W} \sum_{i \in \mathcal{I}_{t\ell}} v_i \phi_b(x_i)(\pi_{ik} - \mathbf{1}[y_i = k]), \tag{9}$$

where $L_{t\ell}$ denotes the unpenalized term in Eq. (8). Different leaves of this tree involve disjoint event sets, so they can be fitted separately while its other-tree offset is held fixed.

The joint forest leaf refit has the same form, but includes every tree in one design matrix. Index its columns by (t, ℓ, b) and define

$$A_{i,(t,\ell,b)} = \mathbf{1}[\ell_t(x_i) = \ell]\, \phi_b(x_i), \qquad \Theta_{(t,\ell,b),k} = \beta_{t\ell k,b}. \tag{10}$$

Here $A \in \mathbb{R}^{N \times (TL_D B)}$ and $\Theta \in \mathbb{R}^{(TL_D B) \times (K-1)}$. Thus $z_{ik} = (A\Theta)_{ik}$ for $k < K - 1$, with the reference logit zero. If $q = (t, \ell, b)$ indexes a design column and $\rho_q = \rho_b$, its objective and smooth gradient are

$$J_{\text{leaf}}(\Theta) = \underbrace{\frac{1}{W} \sum_i v_i \operatorname{CE}(y_i, z_i(\Theta))}_{L(\Theta)} + \lambda_\ell \sum_{q,k} \rho_q |\Theta_{qk}|, \tag{11}$$

$$\nabla_\Theta L = \frac{1}{W} A^\mathsf{T} R, \qquad R_{ik} = v_i(\pi_{ik} - \mathbf{1}[y_i = k]), \quad k < K - 1. \tag{12}$$

Cross-entropy is convex in its logits, the logits are affine in the leaf coefficients, and the L1 term is convex. Both the individual-leaf and joint problems are therefore convex. The joint parameters need not be unique: different trees can contribute compensating logits.

To handle L1 with a smooth bound-constrained solver, flatten the coefficients being fitted into scalars ϑ_r, and let λ_r be their penalty strength ($\lambda_\ell \rho_b$ for a leaf coordinate). For each penalized coordinate, write $\vartheta_r = u_r^+ - u_r^-$ with $u_r^+, u_r^- \geq 0$. This is an exact reformulation because

$$\min_{u^+, u^- \geq 0:\ u^+ - u^- = \vartheta} \lambda_r(u^+ + u^-) = \lambda_r |\vartheta|. \tag{13}$$

Coordinates with $\lambda_r = 0$ are optimized directly as unconstrained real variables. For either smooth leaf loss L, the auxiliary objective and its penalized-coordinate derivatives are

$$\widetilde{J} = L(\vartheta(u)) + \sum_{r:\lambda_r > 0} \lambda_r(u_r^+ + u_r^-), \qquad \frac{\partial \widetilde{J}}{\partial u_r^+} = g_r + \lambda_r, \quad \frac{\partial \widetilde{J}}{\partial u_r^-} = -g_r + \lambda_r, \tag{14}$$

where $g_r = \partial L / \partial \vartheta_r$ is obtained from Eq. (9) or (12). The free-coordinate derivative remains g_r. We minimize this differentiable objective using a bound-constrained quasi-Newton solver and reconstruct the stored coefficients as $\vartheta_r = u_r^+ - u_r^-$.

At a finite iteration budget, both parts can remain positive. Define the actual coefficient objective as $J_{\text{actual}} = L(\vartheta) + \sum_r \lambda_r |\vartheta_r|$. The auxiliary objective exceeds it by the explicit gap

$$\widetilde{J} - J_{\text{actual}} = \sum_{r:\lambda_r > 0} \lambda_r (u_r^+ + u_r^- - |u_r^+ - u_r^-|) = 2 \sum_{r:\lambda_r > 0} \lambda_r \min(u_r^+, u_r^-) \geq 0. \tag{15}$$

Canonical parts $\max(\vartheta_r, 0)$ and $\max(-\vartheta_r, 0)$ remove this gap without changing any logit. We evaluate the candidate with the actual L1 norm and accept it only if its leaf objective does not increase. Since the omitted terms are fixed, this also ensures nonincrease of the full objective in Eq. (6).

3.3 Decision-node optimization

Let $\mathcal{R}_{tp}$ be the training events reaching node p in tree t. Nodes with no reaching events are left unchanged; the following update assumes $\mathcal{R}_{tp} \neq \varnothing$. Keep its ancestors, descendants, leaf functions and all other trees fixed. Ancestors determine $\mathcal{R}_{tp}$, so changing this cut cannot affect any event outside that set. For $i \in \mathcal{R}_{tp}$, let $g_L(x_i)$ and $g_R(x_i)$ be the fixed logit vectors predicted by its two subtrees, and o_i the logit sum from the other trees. The two possible losses are

$$C_i^L = \mathrm{CE}(y_i, o_i + g_L(x_i)), \qquad C_i^R = \mathrm{CE}(y_i, o_i + g_R(x_i)), \qquad \Delta_i = C_i^L - C_i^R. \tag{16}$$

Write $r_i(w,b) = \mathbf{1}[w^\mathsf{T} x_i + b > 0]$ for the proposed right-branch decision. All other penalties and all outside-event losses are constant, so Eq. (6) reduces to

$$Q(w,b) = \frac{1}{W} \sum_{i \in \mathcal{R}_{tp}} v_i \big[(1 - r_i(w,b)) C_i^L + r_i(w,b) C_i^R\big] + \lambda_s(\|w\|_1 + |b|). \tag{17}$$

Define a preferred-branch label $q_i = \mathbf{1}[\Delta_i > 0]$ and an effective weight $\gamma_i = v_i|\Delta_i|$. For either decision $r \in \{0,1\}$,

$$(1-r)C_i^L + rC_i^R = \min(C_i^L, C_i^R) + |\Delta_i|\,\mathbf{1}[r \neq q_i]. \tag{18}$$

Subtracting the first, cut-independent term gives the equivalent weighted misrouting objective

$$Q(w,b) - \frac{1}{W} \sum_{i \in \mathcal{R}_{tp}} v_i \min(C_i^L, C_i^R) = \frac{1}{W} \sum_{i \in \mathcal{R}_{tp}} \gamma_i \mathbf{1}[r_i(w,b) \neq q_i] + \lambda_s(\|w\|_1 + |b|). \tag{19}$$

Thus the reduced labels describe preferred branches, even when the original forest classifies more than two classes. A large $|\Delta_i|$ makes a wrong branch expensive. Events with $\gamma_i = 0$ are indifferent or have zero training weight and give no information for this cut.

Logistic proposal. The discrete misrouting term has no useful ordinary gradient in (w,b). When both branch labels occur among active events, we form a smooth proposal by assigning the right branch probability $\sigma(h_i)$, where $h_i = w^\mathsf{T} x_i + b$ and $\sigma(h) = 1/(1+e^{-h})$. The Bernoulli negative log-likelihood for branch label q_i is

$$-q_i \log \sigma(h_i) - (1-q_i)\log[1 - \sigma(h_i)] = \log(1 + e^{h_i}) - q_i h_i. \tag{20}$$

This gives the weighted proposal objective

$$S(w,b) = \underbrace{\frac{1}{W} \sum_{i \in \mathcal{R}_{tp}:\gamma_i > 0} \gamma_i \big[\log(1 + e^{h_i}) - q_i h_i\big]}_{L_{\log}(w,b)} + \lambda_s(\|w\|_1 + |b|), \tag{21}$$

with smooth derivatives

$$\nabla_w L_{\log} = \frac{1}{W} \sum_i \gamma_i [\sigma(h_i) - q_i] x_i, \qquad \frac{\partial L_{\log}}{\partial b} = \frac{1}{W} \sum_i \gamma_i [\sigma(h_i) - q_i], \tag{22}$$

where these sums run over the same active events as Eq. (21). The L1 penalty includes the bias and can be handled by the positive/negative parameterization above. Let $(\widehat{w}, \widehat{b})$ denote the resulting approximate minimizer. A decrease in S does not imply a decrease in the hard loss Q, so the proposal must be checked against Eq. (17).

Threshold search and acceptance. Let $(w^{(0)}, b^{(0)})$ be the current cut. We consider directions

$$\mathcal{D} = \{w^{(0)}, \widehat{w}, 0\} \cup \{(1-\eta)w^{(0)} + \eta\widehat{w} : \eta \in \{0.25, 0.5, 0.75\}\}. \tag{23}$$

For a fixed direction $d \in \mathcal{D}$, set $s_i = d^{\mathsf{T}}x_i$ and $\tau = -b$, so right routing means $s_i > \tau$. Let $a_1 < \cdots < a_M$ be the distinct projected scores of events in $\mathcal{R}_{tp}$. The intervals

$$I_0 = (-\infty, a_1), \qquad I_m = [a_m, a_{m+1}) \ (1 \le m < M), \qquad I_M = [a_M, \infty) \tag{24}$$

exhaust the possible routings along d. The half-open convention follows from sending equality left. Equal scores move together; the endpoints include all-right and all-left routing.

To evaluate every interval, start with all events on the right and define

$$U = \sum_{i \in \mathcal{R}_{tp}} v_i C_i^R, \qquad B_m = \sum_{i \in \mathcal{R}_{tp}: s_i \le a_m} v_i (C_i^L - C_i^R), \qquad B_0 = 0. \tag{25}$$

The values B_m are obtained by sorting the scores once and accumulating whole tied groups. For $\tau \in I_m$, the objective is

$$Q(d, -\tau) = \frac{U + B_m}{W} + \lambda_s \|d\|_1 + \lambda_s|\tau|. \tag{26}$$

Only the bias penalty varies within an interval. We therefore choose its representable finite threshold nearest zero,

$$\tau_m^* \in \operatorname*{arg\,min}_{\tau \in I_m \cap \mathbb{F}} |\tau|, \tag{27}$$

where $\mathbb{F}$ is the set of finite numbers in the working floating-point format; empty intersections are discarded. This is zero if zero belongs to the interval, the included lower endpoint for an entirely positive interval, and the representable predecessor of the open upper endpoint for an entirely negative interval. The two outer intervals use the same rule. When $\lambda_s = 0$, every threshold in an interval has the same loss; we keep this representative convention.

For each direction d, let $m(d)$ minimize Eq. (26) over its nonempty intervals. We compare the resulting pairs $(d, -\tau^*_{m(d)})$, the raw logistic pair $(\widehat{w}, \widehat{b})$, and the unchanged cut. Each of these proposed cuts is reevaluated using actual hard routing, including both weight and bias penalties. We update only when the best candidate satisfies

$$Q(w^{\text{new}}, b^{\text{new}}) < Q(w^{(0)}, b^{(0)}). \tag{28}$$

Because all omitted terms are fixed, this strictly decreases the full forest objective. The threshold search is exhaustive for each proposed direction; the finite direction set does not exhaust all hyperplanes. In fixed-support refitting, directions must preserve frozen-zero weights, and a frozen-zero bias permits only $\tau = 0$ rather than unrestricted threshold search.

A lower-bound certificate and one-sided labels. During unpenalized fixed-support refitting, $\lambda_s = 0$ and Eq. (19) gives the bound

$$Q(w, b) \ge Q_{\text{lb}} := \frac{1}{W} \sum_{i \in \mathcal{R}_{tp}} v_i \min(C_i^L, C_i^R), \qquad Q(w, b) - Q_{\text{lb}} = \frac{1}{W} \sum_i \gamma_i \mathbf{1}[r_i(w, b) \ne q_i]. \tag{29}$$

The last sum is over $\mathcal{R}_{tp}$. If the current cut chooses the preferred child for every event with $\gamma_i > 0$, the gap is zero. It already minimizes this node's hard subproblem, even over all feasible cuts, so an auxiliary logistic fit is unnecessary. This is a local certificate with subtrees and other trees fixed, not a statement about the full forest optimum.

When active events contain only one branch label, we skip the logistic fit and retain only the current and zero directions in the threshold search. The zero direction gives a constant route whenever the allowed bias permits it. If all $\gamma_i = 0$, any route has the same data loss; the current cut already attains the unpenalized lower bound. These cases introduce no third branch and do not require a finite optimum of a one-class logistic problem.

3.4 Initialization and numerical stopping

Fresh trees are initialized by greedy axis-aligned classification trees fitted to bootstrap draws from the training sample. Subsequent TAO updates use the full training sample. Leaf class counts receive a 0.5 pseudocount before conversion to reference-class log odds; each tree's initial logits are divided by T. Early terminal nodes can be embedded in a complete topology using constant left-routing cuts, which explains some unreachable branches. The sparsity study below continues an existing trained forest.

We distinguish an optimizer iteration, an epoch of alternating tree updates, and convergence of the full run. Reaching an iteration cap alone is not convergence. For fixed routes, let g_q be the gradient of normalized unpenalized cross-entropy with respect to leaf coordinate θ_q, and let λ_q equal λ_ℓ for a penalized feature weight and zero for a bias. The leaf stationarity residual is

$$r_q = \begin{cases} 0, & q \text{ frozen}, \\ |g_q + \lambda_q \operatorname{sign}(\theta_q)|, & q \text{ free}, \ |\theta_q| > \delta, \\ \max(|g_q| - \lambda_q, 0), & q \text{ free}, \ |\theta_q| \le \delta, \end{cases} \qquad r_{\mathrm{KKT}} = \max_q r_q, \tag{30}$$

with numerical zero tolerance $\delta = 10^{-10}$. Here KKT denotes the Karush–Kuhn–Tucker stationarity conditions. Frozen coordinates are subject to equality constraints and do not require zero gradients. The tolerance δ is used only for this residual; support selection uses exact zeros.

We require $r_{\mathrm{KKT}} \le 10^{-6}$, successful inner solves, relative objective gain at most 10^{-5} and a routing-change fraction at most 10^{-4} for three consecutive epochs, after at least five epochs. An event counts as rerouted if its leaf changes in any forest member. The relative gain is $\max(0, (J_{\mathrm{before}} - J_{\mathrm{after}})/\max(|J_{\mathrm{before}}|, 10^{-12}))$; increases above 10^{-8} are rejected. These conditions concern numerical stationarity of leaf fits and the available cut proposals. They do not establish global optimality of the forest.

Implementation and reproducibility

Objective and leaf solvers. src/tao/hard.py contains `cross_entropy_rows`, `ce_value_gradient`, `fit_leaf_parameters`, `HardTAOClassifier._penalty`, `objective`, `_joint_design` and `global_refit`. The joint CSR design activates a leaf block in each tree: a linear-leaf row has $T(F+1)$ stored entries. For 531,200 events and three trees this is 27,091,200 entries. Frozen feature columns are still stored. The solver is SciPy L-BFGS-B. `fit_l1_split` also supports scikit-learn LIBLINEAR for ordinary penalized proposals, with $C = 1/(\lambda_s W)$ and a penalized synthetic intercept. Zero-penalty and masked proposals use L-BFGS-B.
Cuts and epochs. `best_threshold` in src/tao/split_search.py implements the threshold search; `FrozenSupportTAO._split_update` in src/tao/support.py handles fixed support and pointwise certificates. `begin_epoch`/`step_epoch` in src/tao/epoch.py serialize a bottom-up epoch and cache other-tree logits. `initialize_cart` in src/tao/initialization.py initializes fresh trees. `append_zero_tree` can extend a forest while preserving predictions; the selected compact forest was not obtained with that option. `leaf_kkt_residual` and `convergence_check` in src/tao/convergence.py implement Eq. (30) and the stopping tests.

4 Selecting support and refitting

4.1 Select–Freeze–Refit

L1 regularization both removes coefficients and shrinks surviving values. Once a coefficient will be retained, shrinkage can unnecessarily reduce accuracy. Motivated by post-selection refitting [3], we use the following procedure, termed Select–Freeze–Refit (SFR):

1. **Select.** Follow split-only, leaf-only or joint L1 paths using converged terminal fits as warm starts. Choose a support strategy on validation.
2. **Freeze.** Copy the selected parent, fix its topology, and record $M_q = \mathbf{1}[\theta_q \neq 0]$ for every coefficient, including biases.
3. **Refit.** Remove both L1 penalties. First optimize the surviving leaf coefficients with cuts fixed; then optionally alternate cut and leaf updates while preserving the original masks.

Let $\mathcal{C}$ denote coordinates of input-independent cuts and $\theta_{\mathcal{C}}^{\text{parent}}$ their parent values. The full refit minimizes

$$\frac{1}{W}\sum_i v_i \,\mathrm{CE}(y_i, z(x_i;\theta)) \quad \text{subject to } \theta_q = 0 \text{ if } M_q = 0, \quad \theta_{\mathcal{C}} = \theta_{\mathcal{C}}^{\text{parent}}. \tag{31}$$

Free coefficients may change sign or pass through zero; their masks are not reselected. Frozen-zero coefficients cannot regrow. Cuts that were already input-independent retain their parameters and constant decision. A frozen-zero cut bias also remains zero during threshold search. This differs from simply lowering L1 on an unrestricted forest, which could restore removed parameters.

Each stage begins with a joint fixed-route leaf fit; the full stage repeats it after each epoch of cut updates. Model selection uses the declared validation working point among eligible saved fits. A run that eventually converges can have its best validation score at an earlier epoch; the selected snapshot need not itself be stationary.

4.2 Separation and cut-scale ambiguity

Unpenalized logistic fits can exhibit complete or quasi separation, with the loss approaching an infimum as coefficients diverge. With routes fixed, logits are linear in free leaf coordinates. We search for a free direction d such that every true-class margin against another class has a nonnegative change and at least one has a positive change:

$$\frac{d}{d\alpha}\left[z_{y_i}(x_i;\theta+\alpha d) - z_k(x_i;\theta+\alpha d)\right] \geq 0 \quad (v_i > 0,\ k \neq y_i). \tag{32}$$

These are linear inequalities in d. A box-constrained linear program fixes the scale of the direction and searches for a positive summed improvement. A positive feasible result indicates a separating direction; an ambiguous result or timeout is held as inconclusive. Such stages are excluded from selection, while a separately completed leaf-only stage can remain eligible. This diagnostic does not establish uniqueness of the fitted coefficients.

Cut scale presents a separate issue. For any $c > 0$, in exact arithmetic,

$$\mathbf{1}[w^{\mathsf{T}}x + b > 0] = \mathbf{1}[cw^{\mathsf{T}}x + cb > 0], \qquad \|cw\|_1 + |cb| = c(\|w\|_1 + |b|). \tag{33}$$

The hard-routing loss is unchanged by positive rescaling, while raw-coefficient L1 decreases as c decreases. The logistic proposal fixes a scale within its smooth subproblem, but the outer hard objective remains scale-ambiguous. Thus L1 is an empirical sparsity control in this alternating procedure, not an invariant measure of tree size. Small nonzero cut coefficients cannot simply be discarded: their ratios may still define an important boundary. Explicit rescaling before quantization requires verification near boundaries. An input-independent cut can instead be replaced by its Boolean branch choice.

Implementation and reproducibility

Masks and separation. `freeze_support`, `continue_full_refit`, `FrozenSupportTAO` and `separation_check` are in `src/tao/support.py`. They check topology, shapes, finite values, mask hashes and absence of zero regrowth. Leaf-only cuts remain bit-identical. The separation

LP uses all positive-weight training events, checks feasibility, and treats scores above 10^{-7} as positive and at or below 10^{-10} as near-zero; intermediate values and 600-second timeouts are held. Certificates are cached only while routes remain unchanged.
A concrete precision trap occurs in the compact model: scientific node $(t, p) = (2, RR)$ (storage tree 1, node 6) has zero weights and bias $2^{-1074} > 0$. It always chooses right. Rounding the bias to zero changes the decision; folding the constant branch preserves it without representing a subnormal constant in fixed-point arithmetic.

5 Jet-classification study

5.1 Data and comparison protocol

We use 16 high-level jet features from the dataset catalogued as OpenML 42468 [4]. Class 0 combines quark and gluon jets; class 1 combines W, Z and top jets. The fixed partitions contain 531,200 training, 132,800 validation and 166,000 test events, using split seed 42. Validation contains 52,511 background and 80,289 signal events. Each feature is standardized by its training mean μ_f and standard deviation $s_f > 0$:

$$x_f = (r_f - \mu_f)/s_f, \qquad f = 1, \ldots, F, \tag{34}$$

where r_f denotes the unstandardized feature. All models use the same partitions, labels and feature order.

The primary metric is signal efficiency ϵ_s at background efficiency $\epsilon_b = 0.01$, interpolated on the empirical receiver operating characteristic (ROC). AUC denotes its area. Both use the predicted signal probability. For a threshold τ, the operating efficiencies are the fractions of signal and background events accepted by a score above τ. Interpolation can give a point not exactly realized by a strict threshold. With the validation sample here, a realizable threshold accepting at most 1% background permits at most $\lfloor 0.01 \cdot 52{,}511 \rfloor = 525$ background events.

All oblique scans, SFR choices and results below use validation, with AUC secondary to the low-background working point. The test partition was used in an earlier XGBoost reproduction check, but not to select the oblique models or penalties. Reported uncertainties consequently concern validation and condition on the selected fitted models.

The XGBoost reference is an additive ensemble of axis-aligned trees with constant leaves, trained using gradient/Hessian information for greedy splits and regularized leaf updates [5]. It has 100 trees, maximum depth 4, learning rate 0.1, row and feature subsampling 0.8, histogram-based splits, binary logistic objective and random seed 42. Its realized structure contains 1,499 thresholds, 1,599 leaf values and one base bias: 3,099 scalar parameters. Feature indices and topology are excluded, consistently with the oblique count.

Implementation and reproducibility

The project repository [6] contains the archived data contract and training configuration `configs/bdt_qg_vs_wzt_seed42.yaml`; `src/train_bdt.py` trains the reference. The input files are `data/qg_vs_wzt_standardized.npz` and `data/qg_vs_wzt_metadata.json`. `scripts/audit_tao_splits.py` checks split identities. The data hash is

`f5ec6f7ed442010656a904700f8f6421cc34b674a1ec810c5cf6b2de003ed926`

No new split is drawn from OpenML. The root README documents the historical XGBoost test reproduction. `src/evaluate.py` supplies the common ROC interpolation. XGBoost version 3.2.0 is used with one thread; remaining settings are those of that version. The handed-over native JSON fixes the actual model. The paper input generator evaluates validation only.

5.2 Selected models and fixed-support study

Table 1 compares three oblique configurations and XGBoost. All oblique entries use depth-four trees and linear leaves. We call the 418-coefficient three-tree model *Compact refit*; the 837-coefficient model is the *higher-efficiency* alternative. A one-tree reference illustrates the distinction between allocated and retained parameters. Figure 1 shows the corresponding ROCs.

Table 1: Validation comparison. Layout gives tree count and maximum depth (D4 means depth four). Dense and nonzero counts include biases. CPU is median bulk time per event in microseconds, measured as described in the text.

Model	Layout	P_{dense}	P_{nz}	ϵ_s [%]	AUC	CPU [μs]
Compact refit	3 × D4	1581	418	56.20	0.92733	0.974
Higher efficiency	3 × D4	1581	837	57.12	0.92967	1.131
One-tree reference	1 × D4	527	452	55.63	0.92704	0.412
XGBoost	100 × D4	3099	—	55.48	0.92056	1.083

The Compact refit uses 49% fewer allocated scalars than XGBoost and about half the nonzero coefficients of the higher-efficiency oblique model. These are different comparisons: a coefficient slot is not necessarily nonzero, and an oblique feature weight does not have the same hardware cost as an axis-aligned threshold. The one-tree reference has fewer allocated slots but more nonzero coefficients than Compact refit.

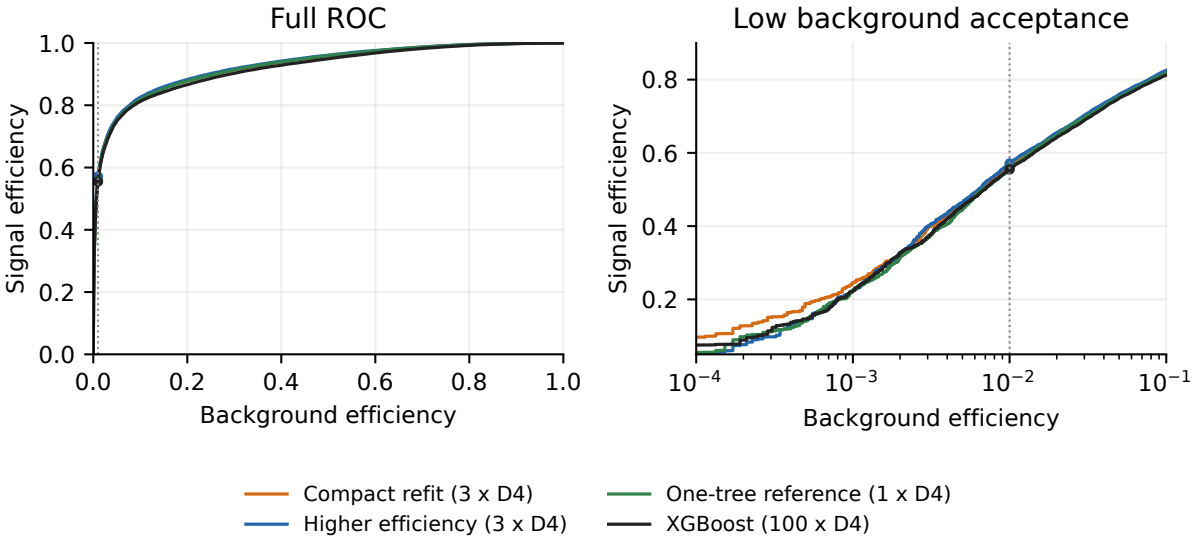


Figure 1: Validation ROCs of the four selected models. The right panel resolves low background acceptance; dots mark the interpolated 1% background point.

For Compact refit, a converged three-tree parent was continued along a split-penalty path $\lambda_s = 3{\cdot}10^{-5}, 10^{-4}, 3{\cdot}10^{-4}, 10^{-3}, 3{\cdot}10^{-3}$, with $\lambda_\ell = 10^{-5}$. The terminal fit at the final penalty supplied 418 coefficients. Leaf-only refitting raised validation efficiency from 54.59% to 54.81%. Full alternating refitting converged after 15 epochs at 55.98%; its best saved validation snapshot was epoch 8 at 56.20%. The selected model is this earlier snapshot, with the same 418 coefficients. The one-tree reference was trained without L1 and without SFR.

Table 2 and Fig. 2 summarize ten parent supports. Six full refits completed, two were held for separation and two for an inconclusive linear-program timeout. The higher-efficiency model is a completed leaf-only refit at 57.12%, improving its own parent by about 0.21 percentage points.

Its full stage was held and is excluded from selection. Stronger penalization does not consistently improve the accuracy–size tradeoff; refitting cannot restore a direction removed by support selection.

Table 2: Fixed-support refitting study. Efficiencies are percentages at 1% background. All models have three depth-four trees except the starred one-tree case. Parent and leaf columns are terminal stage results; best is the best eligible saved leaf/full refit. Held full stages contribute no candidates. The support budget is the parent's exact nonzero count.

λ_s/λ_ℓ	Support	Parent	Leaf	Best	Full stage
0.0001/0.0001	520	55.58	55.82	56.49	complete
0.0003/1e-5	675	56.98	56.74	56.74	timeout
1e-5/3e-5	837	56.91	57.12	57.12	timeout
0.0003/0.0003	394	54.48	54.85	54.85	separated
1e-5/1e-5*	403	55.20	55.06	55.08	complete
0.001/1e-5	561	56.06	56.32	56.46	complete
3e-5/3e-5	777	56.62	56.75	56.75	complete
1e-5/1e-5	889	56.40	56.45	56.74	complete
0.003/1e-5	418	54.59	54.81	56.20	complete
0.001/0.001	317	51.27	52.52	52.52	separated

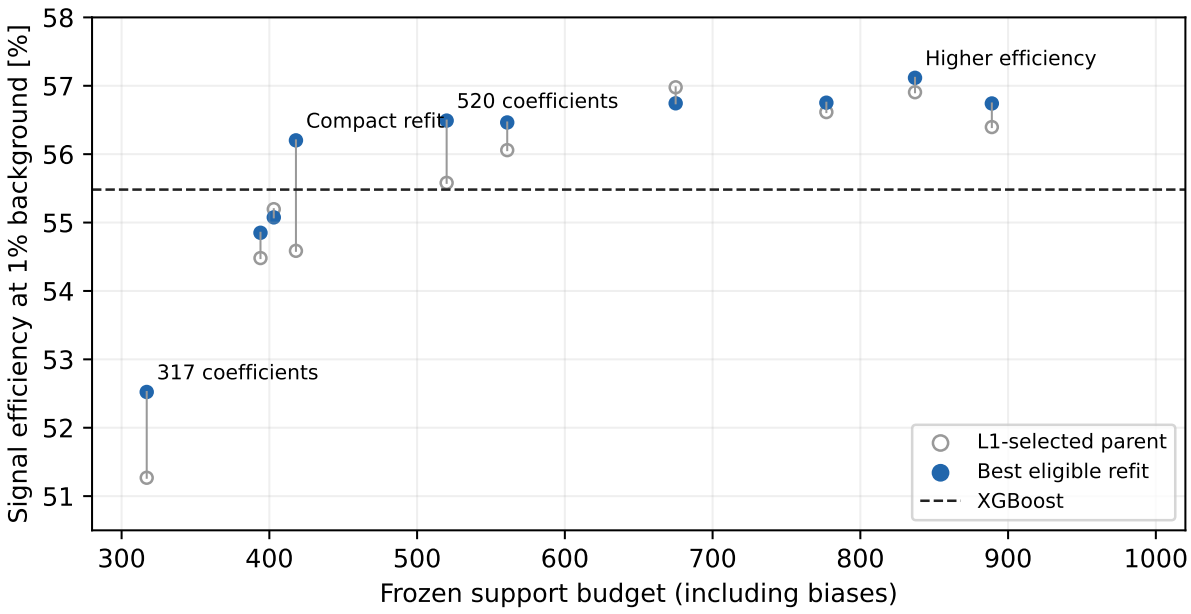


Figure 2: L1-selected terminal parents and their best eligible fixed-support refits. Each vertical segment keeps the support budget fixed. Only refits from eligible stages contribute; the penalty settings are given in Table 2.

5.3 Constant-leaf forests

Constant predictions simplify the hardware leaf stage and permit comparison with scalar-leaf FPGA tree architectures [7]. To study the accuracy cost, we scan $T \in \{1, 3, 5\}$ trees, $D \in \{2, 3, 4\}$ and $\lambda_s \in \{0, 10^{-7}, 10^{-5}\}$, giving 27 configurations. Each uses the same CART initialization seed, training partition and numerical stopping rules as the factorial architecture study. Constant leaves have no feature weights, so their nominal leaf penalty does not affect Eq. (6). We select the saved checkpoint with the highest validation efficiency at 1% background from each converged source run, and then select among penalties for each layout.

All 27 source runs have converged. Table 3 reports the best selected penalty per layout. The interpolated efficiency follows the common ROC convention. We also report a strict threshold on the binary64 signal margin that accepts at most 525 background events. This distinction matters for shallow constant trees: a single depth-D tree has at most 2^D score levels, and even its highest level can exceed the background budget. Connecting ROC vertices then gives an interpolated efficiency that a deterministic threshold cannot realize.

Table 3: Constant-leaf forests: the best eligible penalty per layout. ϵ_s is interpolated at 1% background; $\epsilon_s^{\text{strict}}$ uses a realizable strict margin threshold with background efficiency at most 1%. Both coefficient counts include biases.

Layout	λ_s	P_{dense}	P_{nz}	ϵ_s [%]	$\epsilon_s^{\text{strict}}$ [%]	AUC
1 × D2	10^{-7}	55	55	14.98	0.00	0.88179
1 × D3	10^{-5}	127	126	38.61	0.00	0.90357
1 × D4	0	271	256	45.36	45.33	0.91247
3 × D2	10^{-5}	165	129	30.72	0.00	0.90087
3 × D3	0	381	336	49.01	38.81	0.91763
3 × D4	0	813	753	53.54	53.30	0.92318
5 × D2	10^{-7}	275	275	47.77	47.01	0.91283
5 × D3	10^{-7}	635	615	50.94	50.31	0.92176
5 × D4	0	1355	1250	55.40	54.75	0.92610

The selected constant configuration has 5 trees of depth 4, with 1250 nonzeros in 1355 allocated slots. It reaches 55.40% interpolated signal efficiency; its realizable strict-threshold efficiency is 54.75%. Figure 3 compares the depth-four constant forests with XGBoost and the sparse linear-leaf Compact refit. These are comparisons of selected configurations: their penalty and refitting histories differ, so the comparison is not a leaf-type ablation at identical support.

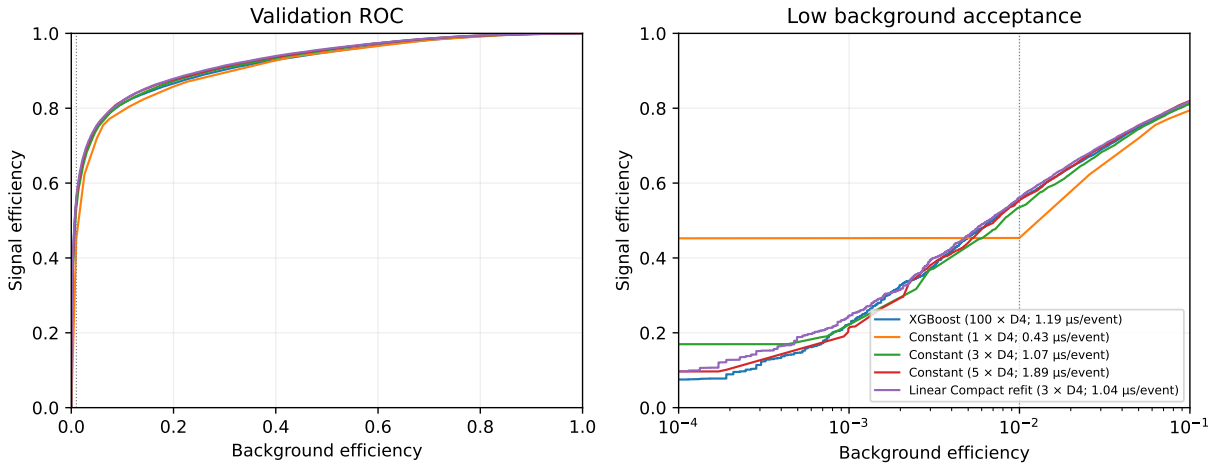


Figure 3: Validation ROCs for the best completed constant-leaf configuration at each depth-four layout, compared with XGBoost and linear-leaf Compact refit. The low-background panel exposes the coarse score steps of small constant forests. Eligibility requires convergence of the source run; selection can choose an earlier saved epoch. Parenthetical times are bulk CPU wall time per event on the 132,800-event batch: one thread, median of nine repeats. TAO uses the NumPy prediction API on binary64 inputs and XGBoost uncached compiled prediction on preconverted binary32 inputs. These follow-up measurements are separate from Table 1 and are not FPGA or single-event latency.

For these candidates, linear predictions reduce the number of nonzero coefficients from 1250 to

418, a factor of 2.99, while increasing the interpolated signal efficiency from 55.40% to 56.20%. After eliminating only input-independent branches and exact-zero terms, the respective all-node multiplication counts are 1095 and 327; selected-path upper bounds are 275 and 80. The additional leaf arithmetic is therefore more than offset by the sparser, smaller forest in this comparison. These scalar operation counts do not determine synthesized resource use: precision, scheduling, constant-multiplier optimization and storage layout must also be specified.

A matched compiled sparse traversal provides an additional CPU check. With one thread and the full 132,800-event validation batch, the median of nine repeats after one warmup is 0.510 microseconds per event for the constant forest and 0.181 for the linear forest. Both use binary64 arithmetic and the same traversal strategy; input conversion and file I/O are excluded. These are CPU batch measurements, with no inference about FPGA latency.

For an exploratory reduced-precision comparison, each cut is first multiplied by a positive power of two, $w'_{tv} = 2^{k_{tv}} w_{tv}$ and $b'_{tv} = 2^{k_{tv}} b_{tv}$. This preserves its real-arithmetic decision; the exported binary64 scaling is constrained to retain every nonzero coefficient, and validation leaf assignments are checked before quantization. We use the nearest-even, saturating quantizer of Eq. (35), with 10 integer bits for inputs, thresholds and cut accumulators, 6 for weights, and 16 for scores and leaf biases, all including sign. Every type uses the same fractional width F. Cast inputs and coefficients precede multiplication; cut products accumulate in feature order, with a cast to the cut-accumulator format after each addition. For a linear leaf, products accumulate similarly in the score format, followed by the leaf bias. The negative tree outputs are summed in the score format. Integer widths are conservatively checked against absolute partial-sum bounds in the observed training box; they have not been minimized separately for the two candidates.

Table 4: Common-format fixed-point emulation of the two selected leaf architectures. Efficiencies are percentages at 1% background. Total widths are $F + 10$ input/threshold bits, $F + 6$ weight bits and $F + 16$ score bits.

Leaf type	F	ϵ_s [%]	$\epsilon_s^{\text{strict}}$ [%]
Constant	18	55.40	54.75
Constant	12	55.40	54.75
Constant	8	55.18	54.38
Linear	18	56.20	56.20
Linear	12	56.21	56.21
Linear	8	56.00	56.00

At $F = 12$, both architectures remain within 0.02 percentage points of their floating-point interpolated working-point efficiency (Table 4). For constant leaves, independent fixed-point path traversal and the existing weighted-cut backend agree exactly on all validation scores. The generated all-node HLS C++ implementation also agrees exactly on a 512-vector check at $F = 12$. The linear-leaf evaluation uses an independent affine-leaf reference; a corresponding HLS backend extension and synthesis remain to be completed. These checks establish numerical feasibility at common precision, rather than an optimized FPGA resource comparison.

Implementation and reproducibility

Controlled cohort and exact snapshots. The constant subset of `configs/tao_hyper_scan_v1.json` comprises cases 3–5, 9–11, 15–17, 21–23, 27–29, 33–35, 39–41, 45–47 and 51–53. These use CART initialization and hard-threshold search. They are separate from the earlier seven-layout, randomly initialized constant study. `scripts/build_constant_leaf_study.py` re-evaluates eligible checkpoints, archives them under `models/constant_study_v1_selected/<sha256>.pkl`, and records selected and terminal epochs, metrics, configurations, convergence checks and hashes in `paper/sources/constant_leaf_study.json`. The selected constant model is

also exported in `paper/sources/constant_leaf_winner.json`; this does not alter the v3 handover or its default. Its selected epoch is 23, while the source run terminates at epoch 36.
Regenerate with the pinned training environment:

```
PYTHONPATH=src:. python scripts/build_constant_leaf_study.py
# Final gate: exits unsuccessfully while any source case is incomplete.
PYTHONPATH=src:. python scripts/build_constant_leaf_study.py --require-complete
```

The two-candidate entry point is `fpga_handoff/CURRENT_CANDIDATES.json`. The constant winner is frozen in `fpga_handoff/leaf_competition_v1/constant_5t_d4/`; the linear candidate points to the unchanged v3 Compact refit. Both input spaces have canonical JSON, hexadecimal coefficients and golden vectors. `docs/leaf_competition_v1/README.md` records CPU/precision results and `scripts/run_leaf_competition.sh` reproduces them in the pinned EL9 container. The scalar-leaf adapter uses unmodified conifer 1.11; the CMSSW 1.7 external cannot evaluate oblique weights. The affine-leaf reference is `fpga/leaf_competition/emulate.cc`, not a conifer HLS backend extension. `scripts/verify_fpga_handoff.py` checks both candidates and both input spaces without a Python training stack.
Training compute times and both probability-based and binary64-margin strict working points are retained in the snapshot and `docs/constant_leaf_study_v1.md`. Incomplete cases remain visible and are never labelled converged or promoted into the table.

5.4 A fixed-point XGBoost reference

As a first numerical implementation check, we emulate the XGBoost reference with signed fixed-point inputs, thresholds and scores. A format with total width W_b and integer width I_b has $F_b = W_b - I_b$ fractional bits and representable range $[-2^{I_b-1}, 2^{I_b-1} - 2^{-F_b}]$. Define its quantizer by

$$Q_{W_b,I_b}(u) = \mathrm{clip}_{W_b,I_b}\left(2^{-F_b}\,\mathrm{round}_{\mathrm{even}}(2^{F_b}u)\right), \tag{35}$$

where rounding chooses the nearest integer with ties to even, and clipping saturates at the format's endpoints. We quantize inputs and thresholds before evaluating the strict-left XGBoost cuts. Let $s_{t\ell}$ denote a scalar leaf contribution with the learning rate absorbed, and s_0 the initial margin. These values use the score format; tree scores are combined by a balanced adder tree with casts to that format after additions. Its result is added to the quantized initial margin and cast once more. No sigmoid is needed for selection.

The maximum absolute training input is 8.96 and the largest absolute tree threshold is 4.73, so we use $I_b = 5$ for inputs and thresholds. For the $T = 100$ tree scores, define

$$A = |s_0| + \sum_{t=1}^{T} \max_{\ell} |s_{t\ell}| \simeq 15.09. \tag{36}$$

At the coarsest fractional width $F_{\min} = 4$, rounding the T leaf values and initial margin can add at most $(T+1)2^{-F_{\min}-1}$ to this bound. Their quantized sum and all partial sums therefore fit within an absolute bound of 18.25, covered by $I_b = 6$ for scores. With no overflow, sums of values on this common fixed-point grid introduce no further fractional rounding. No validation input lies outside the chosen input range. We vary the common fractional width while keeping these integer widths fixed. Table 5 shows the resulting validation performance. The 17/18-bit input/score pair retains the reference working-point efficiency; the coarsest 9/10-bit pair loses performance and produces more substantial score ties. Small positive changes in the interpolated point are not interpreted as an established accuracy gain.

Independent C++ executions agree exactly for the tested fixed-point outputs. This establishes numerical agreement of the emulators; FPGA resource use, latency and initiation interval require synthesis for a specified device, clock and interface.

Table 5: XGBoost fixed-point emulation on the same validation partition. Widths include the sign bit; input and threshold formats are identical. The native reference has $\epsilon_s = 55.48\%$ and AUC 0.92056. The strict-threshold column uses background efficiency at most 1%.

Input bits	Score bits	ϵ_s [%]	$\epsilon_s^{\text{strict}}$ [%]	AUC
23	24	55.48	55.48	0.92056
17	18	55.49	55.49	0.92056
15	16	55.46	55.45	0.92056
13	14	55.56	55.53	0.92048
11	12	55.47	55.29	0.92040
9	10	52.05	50.85	0.91760

Implementation and reproducibility

Tested chain. `docs/conifer_xgboost_v1/README.md` specifies the EL9 container, CMSSW workspace, dependency pins and commands. Conversion and HLS C++ generation use conifer 1.11; the axis-aligned reference is also evaluated with the actual CMSSW conifer 1.7 external. Its bridge rejects oblique weights. The full validation compares 13,280,000 native/exported leaf IDs, plus boundary checks. Each of eight C++ profiles is checked on 512 HLS vectors, including all 324 frozen XGBoost handover vectors. Every fixed-point comparison is exact. `fpga/conifer/compat.py` reconciles native binary32 thresholds, leaf values, initial margin and actual depth; upstream conversion is retained as a diagnostic. It is important to preserve XGBoost's input rounding and strict-left equality convention. The score-type settings in the table are `AP_RND_CONV,AP_SAT`; axis weights are exact one-hot constants.
`docs/conifer_xgboost_v1/validation.json` contains full provenance and per-profile metrics. `scripts/validate_xgboost_conifer.py` reads training inputs for range selection and validation arrays for evaluation; test arrays are not loaded. Generated projects use conifer's default device/clock only as export placeholders. No FPGA synthesis has been run, and CPU timings that include file I/O are not reported as hardware latency.

5.5 Uncertainty and inference cost

A paired class-stratified bootstrap with 1,000 replicates, seed 20261025, recomputes the interpolated working point in each replicate. Table 6 gives percentile intervals for the difference from XGBoost. The Compact interval includes zero. These intervals condition on the selected models and account for neither hyperparameter/checkpoint selection nor training-seed variation; they are not independent test claims.

Table 6: Conditional validation efficiency difference from XGBoost, in percentage points (pp), with paired 95% percentile intervals.

Model	Difference [pp]	Interval [pp]
Compact refit	+0.72	[-0.43, +1.52]
Higher efficiency	+1.63	[+0.65, +2.55]
One-tree reference	+0.15	[-1.08, +0.97]

Table 7 gives sparse arithmetic bounds. For comparison, a dense three-tree depth-four linear forest has 240 selected-path or 1,488 all-node feature multiplications and 12 path decisions. The CPU timings in Table 1 use 132,800-event validation batches, one thread, nine repeats and the median wall time per event. They compare a NumPy oblique traversal with compiled XGBoost and are

bulk-throughput measurements. Parameter and operation counts motivate implementation studies; FPGA cost requires a numeric format, device, clock and synthesis strategy.

Table 7: Sparse arithmetic bounds after removing input-independent unreachable branches. Path counts bound one selected route; all-node counts evaluate every reachable affine function. Bias additions are excluded from multiplications.

Model	P_{reach}	Path mult.	All-node mult.	Path decisions
Compact refit	379	80	327	12
Higher efficiency	837	163	744	12
One-tree reference	452	65	421	4

Implementation and reproducibility

Selected results. The frozen comparison uses the v3 model bundle. `fpga_handoff/CURRENT_MODEL.json` selects `fpga_handoff/v3/compact_refit_3t_d4/`. The 837-coefficient alternative is `highest_efficiency_3t_d4`; the one-tree reference is `single_tree_1t_d4`. All exact source snapshots and masks are recorded in the handover manifest. A later 57.24% full-refit diagnostic for the higher-efficiency support is held and not an eligible model.

Compact training chain. Start from the v2 `winner_3t_d4` coefficients, run `configs/l1_path_v1/control.json`, then the split path `split_01.json` through `split_05.json` in `configs/l1_path_v1/`. The manifest is `configs/tao_l1_path_v1.json`. The final terminal parent feeds `configs/support_refit_scan_v1/split300.json`. Parent training uses `scripts/train_tao_fully.py`; SFR uses `scripts/train_tao_support_refit.py`, `scripts/run_support_refit_scan_case.py` and `scripts/support_refit_artifacts.py`. Exact refit reproduction begins with the archived hash-pinned terminal parent, rather than a fresh fit at the final penalties. The selected checkpoint hash is

`e89b8923fba66f5df02046e04a1828a8ea4c56f60c1d7b201a2a2f92a0ab00fa`

Budgets. The penalty path starts with local/joint/split iteration budgets 300/250/300 and logs repeated chunks or adaptive increases. Compact refit uses 400/400/800, L-BFGS-B tolerances `ftol=1e-12`, `gtol=1e-8`, at most 100 joint-fit chunks and 200 epochs per stage. A 2,700-second allocation checkpoints with return code 75. A mild-control router required a documented 800-to-4,000 iteration-budget amendment and converged after 2,028 auxiliary iterations; this changed no objective or stopping tolerances. See `scripts/amend_support_solver_budget.py` and `docs/support_refit_scan_budget_amendments.json`.

Research status. These results were frozen on 25 October 2026. The separate dated record `docs/handover_status_2026-10-26.json` found 37/58 architecture cases converged and the 16-point L1 path complete. A running five-tree depth-four checkpoint reached 57.13% with 1,258 nonzeros but was unconverged and was not promoted. `src/tao/uncertainty.py` implements the bootstrap; `scripts/build_paper_inputs.py` regenerates frozen tables and curves. `sources/provenance.json` in this document folder records hashes of inputs, numerical source files, tables and figures.

6 An example learned tree

Actual cuts illustrate the mixture of axis-aligned, oblique and constant nodes. In Compact refit, using the standardized feature indices of Table 8, the first tree contains

$$h_{1,\varnothing}(x) \simeq x_4 + 0.4216204 \tag{37}$$

$$h_{1,L}(x) = 0 \tag{38}$$

$$h_{1,R}(x) \simeq 0.088106595x_4 + 0.0025365055x_6 + 0.17735909x_{11} + 0.031223362x_{15} + 0.10064081x_{16} + 0.019058664 \tag{39}$$

Coefficients marked $\simeq$ are rounded to eight significant digits. At the root an event goes right if $x_4 > -0.4216204$. Otherwise it reaches L, whose constant cut sends it left again. Hence the entire LR subtree is unreachable. The root's right child combines five input features.

For the second tree, another learned cut is

$$h_{2,L}(x) \simeq -2.3776136 \cdot 10^{-11}x_{16} - 3.1703281 \cdot 10^{-11} \tag{40}$$

Its small scale does not make it uninformative: it routes right approximately when $x_{16} < -1.3334$. Setting both coefficients to zero would turn it into a constant left route. This example motivates distinguishing exact sparsity from coefficient magnitude before quantization.

Implementation and reproducibility

Print the actual functions. `TreePrinter` in `src/tao/printing.py` provides `print_tree`, `format_tree`, `format_forest` and `records`. The CLI `scripts/print_tao_tree.py` prints every cut and affine leaf, including unreachable branches. It is read-only and separate from pinned fitting code. For example, after setting the environment described below:

```
import json
from pathlib import Path
from tao.printing import TreePrinter
spec = json.loads(Path(
    "fpga_handoff/v3/compact_refit_3t_d4/standardized/model.json"
).read_text())
TreePrinter(spec, precision=17).print_tree(0)
```

The complete 17-digit record is `generated/compact_forest_full_precision.txt`; the annotated supplement prints all three trees at nine display digits. Only exact-zero terms are omitted. Independent tests parse the printed expressions, recover all 418 nonzero coefficients, count 379 reachable coefficients, and compare predictions with the portable evaluator. They also check equality routing, constant leaves, tiny terms and non-mutation. The full suite passed 81 tests.

7 Discussion

Shallow oblique forests with linear leaves provide competitive validation performance with fewer coefficients than the axis-aligned reference in this application. Compact refit reaches 56.20% signal efficiency at 1% background acceptance, with 418 nonzeros in 1,581 allocated slots. The higher-efficiency alternative reaches 57.12% with 837 nonzeros. The conditional uncertainty on the Compact–XGBoost difference includes zero, so the principal result is a compression point at comparable validation performance.

Fixed-support refitting recovers accuracy lost to shrinkage without restoring removed coefficients. Its benefit depends on the selected support: aggressive L1 can remove useful directions that refitting cannot recover. Separation, cut-scale ambiguity and the distinction between a converged terminal model and a validation-selected earlier snapshot remain relevant to interpreting training results. Axis-aligned cuts are contained in the oblique family, but this does not imply that three shallow trees can represent every function of a hundred-tree ensemble.

The next step is to quantify how the reduced arithmetic workload translates into hardware. This requires explicit feature, coefficient and accumulator formats, verification of routing changes near cut boundaries, calibration of a realizable working-point threshold, and synthesis measurements at a specified device and clock. Further model choices should be completed before additional test evaluation, retaining disclosure of the earlier XGBoost-only test check.

Implementation and reproducibility

Environment and entry points. Training and input regeneration use Python 3.10 with the pinned `requirements.txt`: NumPy 1.26.4, SciPy 1.15.2, scikit-learn 1.7.2, XGBoost 3.2.0, PyYAML 6.0.3 and matplotlib 3.10.7. The reference CLIP checkout is `/users/robert.schoefbeck/CMS/tree_benchmark`; its Conda environment is `tree-benchmark`. On CLIP use the conda-forge CPU XGBoost build, since the PyPI Linux wheel requires newer glibc than the reference login environment. No GPU is needed.

```
source /software/f2022/software/mamba/24.3.0-0/etc/profile.d/conda.sh
conda activate tree-benchmark
cd /users/robert.schoefbeck/CMS/tree_benchmark
export PYTHONPATH="$PWD/src:$PWD" PYTHONDONTWRITEBYTECODE=1
export OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 MKL_NUM_THREADS=1
python scripts/print_tao_tree.py \
  --model fpga_handoff/v3/compact_refit_3t_d4/standardized/model.json \
  --precision 17 --output /tmp/compact_forest.txt
```

For a new training account or checkout, the complete environment-creation and controlled refit recipe is in `docs/DORIAN_HANDOVER.md`. The datasets and pickle archives are deliberately not in Git. Exact reproduction of the refit starts from the hash-pinned terminal parent; rebuilding its entire L1 path can produce differently serialized checkpoint bytes and requires a new explicit source hash/run name. Existing named runs must not be resumed under changed training code or stopping policies. The standalone `paper/` folder, however, contains all generated text/tables and figures needed to compile this PDF without datasets, model pickles or a Python training environment.

Implementation map.

Object / operation	Entry point
Routing, logits, objective	`HardTree`; `HardTAOClassifier` in `src/tao/hard.py`
Weighted leaf optimization	`fit_leaf_parameters`; `ce_value_gradient`
Joint leaf design/refit	`_joint_design`; `global_refit`
Reduced split proposals	`_hard_split_update`; `FrozenSupportTAO._split_update`
Sorted hard threshold search	`best_threshold` in `src/tao/split_search.py`
Node-resumable epoch	`begin_epoch`; `step_epoch` in `src/tao/epoch.py`
Masks and separation	`freeze_support`; `continue_full_refit`; `separation_check`
Stopping and exact-zero work counts	`leaf_kkt_residual`; `convergence_check`; `sparsity_profile`
Read-only equations	`TreePrinter.print_tree` in `src/tao/printing.py`
Portable inference / firmware export	`export_model`; `predict_margin`; `cpp_header` in `src/tao/portable.py`

Frozen coefficients and reference verification. `fpga_handoff/v3/manifest.json` hashes the candidate JSONs, hexadecimal C++ headers, feature contract, conservative training-box ranges and golden vectors. Each candidate has both standardized and raw-input variants; choose one and use its matching vectors and working-point threshold. The default JSON hash is

`0ce4fabc4ca74d3fcd2ddb8c94bb6b614b4f2fb998a9225873e167977922d62a`

The corresponding immutable pickle is named by the checkpoint hash given in the Compact training chain below, under `models/support_refit_scan_v1_selected/` on CLIP. The binary portable JSON and headers suffice for firmware; Python pickles are not needed there.

The C++17 reference in `fpga_handoff/reference.cpp` passed all six v3 variants: 1,594 vector evaluations with exact leaf assignments and maximum margin difference below 2.5×10^{-14}. The verifier `scripts/verify_fpga_handoff.py` needs only Python's standard library and a C++17 compiler. It checks bundle hashes before compiling/running the golden vectors and preserves an existing frozen verification report. Fixed-point verification remains a separate next

step. v1 and v2 are retained historical bundles; `fpga_handoff/CURRENT_MODEL.json` selects v3 Compact refit for new implementation work.

References

[1] Miguel Á. Carreira-Perpiñán and Pooya Tavallali. Alternating optimization of decision trees, with application to learning sparse oblique trees. In *Advances in Neural Information Processing Systems*, volume 31, 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/hash/185c29dc24325934ee377cfda20e414c-Abstract.html.

[2] Suryabhan Singh Hada, Miguel Á. Carreira-Perpiñán, and Arman Zharmagambetov. Sparse oblique decision trees: A tool to understand and manipulate neural net features. arXiv:2104.02922, 2021. URL https://arxiv.org/abs/2104.02922. First posted in 2021; revised manuscript and journal version in 2023.

[3] Nicolai Meinshausen. Relaxed lasso. *Computational Statistics & Data Analysis*, 52(1):374–393, 2007. doi: 10.1016/j.csda.2006.12.019. URL https://people.math.ethz.ch/~nicolai/relaxo.pdf.

[4] OpenML. hls4ml_lhc_jets_hlf, dataset 42468. Dataset catalogue. URL https://www.openml.org/d/42468. The project uses its archived feature arrays and fixed splits, rather than a new split from the catalogue.

[5] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785. URL https://arxiv.org/abs/1603.02754.

[6] HEPHY Analysis Software. Oblique tree benchmark, 2026. URL https://github.com/HephyAnalysisSW/oblique-tree-benchmark. Implementation, frozen v3 FPGA handover and dated research records; exact source fingerprints accompany this note.

[7] Sioni Summers, Giuseppe Di Guglielmo, Javier Duarte, Philip Harris, Duc Hoang, Sergo Jindariani, Edward Kreinar, Vladimir Loncar, Jennifer Ngadiuba, Maurizio Pierini, Dylan Rankin, Nhan Tran, and Zhenbin Wu. Fast inference of boosted decision trees in FPGAs for particle physics. *Journal of Instrumentation*, 15:P05026, 2020. doi: 10.1088/1748-0221/15/05/P05026. URL https://arxiv.org/abs/2002.02534.

A Input feature order

Table 8 defines the coordinates r_f of the archived feature vector, using the source dataset's feature names. All displayed cuts use their standardized counterparts x_f from Eq. (34).

Table 8: Feature-coordinate convention. Names identify columns in the high-level jet dataset [4].

f	Dataset feature	f	Dataset feature
1	`zlogz`	9	`d2_a1_b1_mmdt`
2	`c1_b0_mmdt`	10	`d2_a1_b2_mmdt`
3	`c1_b1_mmdt`	11	`m2_b1_mmdt`
4	`c1_b2_mmdt`	12	`m2_b2_mmdt`
5	`c2_b1_mmdt`	13	`n2_b1_mmdt`
6	`c2_b2_mmdt`	14	`n2_b2_mmdt`
7	`d2_b1_mmdt`	15	`mass_mmdt`
8	`d2_b2_mmdt`	16	`multiplicity`

Implementation supplement: complete forest printout

Implementation and reproducibility

This record is generated by `TreePrinter.format_tree` from the exact v3 standardized model. It lists every internal decision and every leaf output in preorder, with heap indices and root-to-node paths. A node labelled unreachable is excluded only by an input-independent constant ancestor, not by absent training or validation examples. Numeric values are displayed with nine significant digits. The full 17-digit record is checked in as `generated/compact_forest_full_precision.txt`. These listings are for inspection; implementation should consume the frozen JSON or hexadecimal header. In all trees, a leaf output contributes to the background logit; the signal margin is the *negative sum* of the three selected outputs.

Reading the record. A path such as `RLLR` means right, left, left, right from the root. This path is the scientific node or leaf label. The extra `N` and `L` labels are internal-node and leaf-array indices; `heap` is a storage index. They have no mathematical role. Trees are numbered from zero in the file and from one in the scientific text.

Implementation and reproducibility

Tree 0 (scientific tree $t = 1$).

```
TREE 0: depth=4, leaves=linear, input=standardized
Coefficient display: 9 significant digits; exact zeros omitted.
At every cut: h > 0 goes RIGHT; h <= 0 goes LEFT (including equality).

N0 [path=root; heap=0; reachable]
  h_0_0(x) = 1 * c1_b2_mmdt + 0.421620399
  <= 0 -> N1; > 0 -> N2

  N1 [path=L; heap=1; reachable]
    h_0_1(x) = 0
    <= 0 -> N3; > 0 -> N4; CONSTANT: always LEFT

    N3 [path=LL; heap=3; reachable]
      h_0_3(x) = 0
      <= 0 -> N7; > 0 -> N8; CONSTANT: always LEFT

      N7 [path=LLL; heap=7; reachable]
        h_0_7(x) = 0.318308228 * c1_b0_mmdt + 0.330704464
        <= 0 -> L0; > 0 -> L1

        L0 [path=LLLL; heap=15; reachable]
          f_0_0(x) = 0.723115375 * zlogz - 0.424343964 * c1_b0_mmdt - 0.446947898 * c1_b1_mmdt -
↪ 0.415001941 * c2_b1_mmdt - 0.139216682 * d2_b1_mmdt - 0.139216682 * d2_a1_b1_mmdt + 0.54944101 *
↪ m2_b1_mmdt - 0.495073946 * m2_b2_mmdt + 0.059415836 * n2_b1_mmdt + 0.858013396 * multiplicity -
↪ 1.35727329

        L1 [path=LLLR; heap=16; reachable]
          f_0_1(x) = 1.51929697 * zlogz + 0.669978191 * c1_b0_mmdt - 3.56315299 * c1_b2_mmdt +
↪ 0.613457803 * c2_b1_mmdt - 0.396278128 * d2_b1_mmdt + 0.152304141 * d2_b2_mmdt - 0.396278128 *
↪ d2_a1_b1_mmdt - 0.199974866 * m2_b2_mmdt + 0.980332415 * n2_b1_mmdt - 0.420497004 * n2_b2_mmdt +
↪ 0.396719294 * mass_mmdt + 1.34144785 * multiplicity - 1.72085229

      N8 [path=LLR; heap=8; UNREACHABLE (constant ancestor)]
        h_0_8(x) = 0
        <= 0 -> L2; > 0 -> L3; CONSTANT: always LEFT

        L2 [path=LLRL; heap=17; UNREACHABLE (constant ancestor)]
          f_0_2(x) = 0

        L3 [path=LLRR; heap=18; UNREACHABLE (constant ancestor)]
          f_0_3(x) = 0

    N4 [path=LR; heap=4; UNREACHABLE (constant ancestor)]
      h_0_4(x) = 0
      <= 0 -> N9; > 0 -> N10; CONSTANT: always LEFT

      N9 [path=LRL; heap=9; UNREACHABLE (constant ancestor)]
        h_0_9(x) = -0.0491155824 * c1_b0_mmdt + 0.000379329885
        <= 0 -> L4; > 0 -> L5

        L4 [path=LRLL; heap=19; UNREACHABLE (constant ancestor)]
          f_0_4(x) = 0

        L5 [path=LRLR; heap=20; UNREACHABLE (constant ancestor)]
          f_0_5(x) = 0
```

```
      N10 [path=LRR; heap=10; UNREACHABLE (constant ancestor)]
        h_0_10(x) = 1.75931373e-09 * c1_b0_mmdt - 3.92622475e-09 * m2_b2_mmdt + 1.22858604e-07 *
↪ multiplicity - 1.097723e-07
        <= 0 -> L6; > 0 -> L7
        L6 [path=LRRL; heap=21; UNREACHABLE (constant ancestor)]
          f_0_6(x) = 0
        L7 [path=LRRR; heap=22; UNREACHABLE (constant ancestor)]
          f_0_7(x) = 0
  N2 [path=R; heap=2; reachable]
    h_0_2(x) = 0.0881065951 * c1_b2_mmdt + 0.0025365055 * c2_b2_mmdt + 0.17735909 * m2_b1_mmdt +
↪ 0.0312233619 * mass_mmdt + 0.100640807 * multiplicity + 0.0190586642
    <= 0 -> N5; > 0 -> N6
    N5 [path=RL; heap=5; reachable]
      h_0_5(x) = 0
      <= 0 -> N11; > 0 -> N12; CONSTANT: always LEFT
      N11 [path=RLL; heap=11; reachable]
        h_0_11(x) = -9.45167785e-16 * d2_b1_mmdt - 5.84426162e-16
        <= 0 -> L8; > 0 -> L9
        L8 [path=RLLL; heap=23; reachable]
          f_0_8(x) = 0.472450802 * zlogz + 0.0989313467 * c1_b0_mmdt - 0.831430736 * c1_b2_mmdt -
↪ 0.431555721 * c2_b2_mmdt + 0.0150358961 * d2_b1_mmdt + 0.648844514 * d2_b2_mmdt + 0.0150358961 *
↪ d2_a1_b1_mmdt - 3.09641516 * d2_a1_b2_mmdt + 1.49849996 * m2_b1_mmdt + 1.81153643 * n2_b1_mmdt -
↪ 0.893777123 * n2_b2_mmdt + 0.182450792 * multiplicity - 2.83938374
        L9 [path=RLLR; heap=24; reachable]
          f_0_9(x) = 0.232327574 * zlogz - 0.23205911 * c1_b0_mmdt - 0.516471161 * c1_b1_mmdt -
↪ 0.848527306 * c2_b1_mmdt - 1.97477014 * c2_b2_mmdt + 1.42114491 * d2_b2_mmdt + 0.536602443 *
↪ m2_b1_mmdt + 0.651756707 * mass_mmdt - 1.64666395
      N12 [path=RLR; heap=12; UNREACHABLE (constant ancestor)]
        h_0_12(x) = 9.89173599e-11 * c1_b2_mmdt + 1.79437153e-09 * c2_b1_mmdt + 1.81919634e-09 *
↪ c2_b2_mmdt + 1.04854961e-09
        <= 0 -> L10; > 0 -> L11
        L10 [path=RLRL; heap=25; UNREACHABLE (constant ancestor)]
          f_0_10(x) = 0
        L11 [path=RLRR; heap=26; UNREACHABLE (constant ancestor)]
          f_0_11(x) = 0
    N6 [path=RR; heap=6; reachable]
      h_0_6(x) = 0.164748862 * c1_b2_mmdt + 0.185195199 * c2_b2_mmdt + 0.271888733 * d2_a1_b2_mmdt +
↪ 1.06018366 * mass_mmdt - 0.265110937 * multiplicity - 1.32289565
      <= 0 -> N13; > 0 -> N14
      N13 [path=RRL; heap=13; reachable]
        h_0_13(x) = 0.635416434 * c1_b2_mmdt - 0.475835509 * multiplicity + 0.00561843162
        <= 0 -> L12; > 0 -> L13
        L12 [path=RRLL; heap=27; reachable]
          f_0_12(x) = 0.000756449729 * zlogz + 0.576353531 * c1_b0_mmdt + 1.62064775 * c1_b1_mmdt -
↪ 2.48681119 * c1_b2_mmdt - 2.10518311 * c2_b1_mmdt + 1.37970957 * c2_b2_mmdt - 0.0135562553 *
↪ d2_b1_mmdt + 0.338362995 * d2_b2_mmdt - 0.0135562553 * d2_a1_b1_mmdt - 1.72149735 * d2_a1_b2_mmdt -
↪ 0.38025814 * m2_b1_mmdt - 0.351321056 * m2_b2_mmdt + 2.69042188 * n2_b1_mmdt - 0.919390109 *
↪ n2_b2_mmdt + 0.32251432 * mass_mmdt + 0.582538651 * multiplicity - 3.45473882
        L13 [path=RRLR; heap=28; reachable]
          f_0_13(x) = -0.140117186 * c1_b2_mmdt + 0.652505422 * c2_b1_mmdt + 0.253434919 * d2_b1_mmdt -
↪ 0.602112106 * d2_b2_mmdt + 0.253434919 * d2_a1_b1_mmdt - 0.527210682 * d2_a1_b2_mmdt - 0.00193453091 *
↪  n2_b2_mmdt - 1.64156297 * mass_mmdt + 0.0192123329 * multiplicity - 2.68453555
      N14 [path=RRR; heap=14; reachable]
        h_0_14(x) = 0
        <= 0 -> L14; > 0 -> L15; CONSTANT: always LEFT
        L14 [path=RRRL; heap=29; reachable]
          f_0_14(x) = -0.727395772 * zlogz - 1.0021119 * c1_b0_mmdt - 1.77913852 * c1_b1_mmdt +
↪ 1.29863387 * c1_b2_mmdt - 5.22419531 * c2_b1_mmdt + 1.67449112 * c2_b2_mmdt + 0.609159654 *
↪ d2_b1_mmdt - 1.14240277 * d2_b2_mmdt + 0.609159654 * d2_a1_b1_mmdt + 0.462711749 * d2_a1_b2_mmdt -
↪ 0.379623445 * m2_b1_mmdt - 1.49230987 * m2_b2_mmdt + 1.44153754 * n2_b1_mmdt - 4.67940637 * mass_mmdt
↪  + 1.29820647 * multiplicity + 5.68806483
        L15 [path=RRRR; heap=30; UNREACHABLE (constant ancestor)]
          f_0_15(x) = 0
Selected leaf f is a class-0 logit contribution; forest signal margin = -sum(f).
```

Implementation and reproducibility

Tree 1 (scientific tree $t = 2$).

```
TREE 1: depth=4, leaves=linear, input=standardized
Coefficient display: 9 significant digits; exact zeros omitted.
At every cut: h > 0 goes RIGHT; h <= 0 goes LEFT (including equality).

N0 [path=root; heap=0; reachable]
  h_1_0(x) = 1 * c1_b2_mmdt + 0.421515048
  <= 0 -> N1; > 0 -> N2

  N1 [path=L; heap=1; reachable]
    h_1_1(x) = -2.3776136e-11 * multiplicity - 3.17032814e-11
    <= 0 -> N3; > 0 -> N4

    N3 [path=LL; heap=3; reachable]
      h_1_3(x) = -0.0032891119 * zlogz + 0.0142175742 * d2_b1_mmdt + 0.00121827303 * d2_a1_b1_mmdt -
↪ 0.000244599886 * m2_b2_mmdt - 0.00553888952 * n2_b2_mmdt + 0.295132576 * mass_mmdt + 0.0218087714
      <= 0 -> N7; > 0 -> N8

      N7 [path=LLL; heap=7; reachable]
        h_1_7(x) = 0.0213173363 * zlogz + 0.612901764 * c1_b0_mmdt - 0.0606512036
        <= 0 -> L0; > 0 -> L1

        L0 [path=LLLL; heap=15; reachable]
          f_1_0(x) = 1.02493155 * zlogz + 0.475780074 * c1_b0_mmdt - 3.46865641 * c1_b1_mmdt +
↪ 1.39660226 * c1_b2_mmdt + 0.0808524935 * d2_b1_mmdt + 0.0625866107 * d2_b2_mmdt + 0.0808524935 *
↪ d2_a1_b1_mmdt + 0.265262813 * d2_a1_b2_mmdt + 0.0469595498 * m2_b1_mmdt - 0.00734786374 * m2_b2_mmdt +
↪  3.49945192 * mass_mmdt - 0.346146245

        L1 [path=LLLR; heap=16; reachable]
          f_1_1(x) = 2.42880675 * zlogz + 3.52193453 * c1_b0_mmdt - 2.27198818 * c1_b1_mmdt - 0.640806627
↪  * c1_b2_mmdt + 0.0986683506 * d2_b1_mmdt + 0.101873458 * d2_b2_mmdt + 0.0986683506 * d2_a1_b1_mmdt +
↪  0.331194728 * m2_b1_mmdt + 0.0539714566 * n2_b1_mmdt + 3.24454093 * mass_mmdt + 0.503474228 *
↪ multiplicity - 1.50898053

      N8 [path=LLR; heap=8; reachable]
        h_1_8(x) = 7.32169007e-19 * zlogz - 2.06778204e-19 * d2_b1_mmdt - 1.91895639e-18 * d2_a1_b1_mmdt
↪  + 4.16333634e-17 * mass_mmdt + 5.02329003e-19
        <= 0 -> L2; > 0 -> L3

        L2 [path=LLRL; heap=17; reachable]
          f_1_2(x) = 0.398116558 * d2_b1_mmdt + 0.398116558 * d2_a1_b1_mmdt - 0.660830844 *
↪ d2_a1_b2_mmdt - 0.114827421 * m2_b2_mmdt - 0.310979867 * n2_b2_mmdt + 0.846928396 * multiplicity -
↪ 0.9437156

        L3 [path=LLRR; heap=18; reachable]
          f_1_3(x) = 0.244294729 * zlogz + 0.880510729 * d2_b2_mmdt - 1.42081066 * m2_b1_mmdt +
↪ 1.71265082 * multiplicity - 2.14718376

    N4 [path=LR; heap=4; reachable]
      h_1_4(x) = 0
      <= 0 -> N9; > 0 -> N10; CONSTANT: always LEFT

      N9 [path=LRL; heap=9; reachable]
        h_1_9(x) = 0.128095519 * m2_b1_mmdt + 0.00063403479 * multiplicity - 0.0832689733
        <= 0 -> L4; > 0 -> L5

        L4 [path=LRLL; heap=19; reachable]
          f_1_4(x) = 0.232595756 * zlogz + 0.336338146 * c1_b0_mmdt + 0.186230458 * d2_b1_mmdt +
↪ 0.186230458 * d2_a1_b1_mmdt - 0.0975771333 * m2_b1_mmdt + 0.100432843 * m2_b2_mmdt - 0.712895383 *
↪ multiplicity - 1.51109101

        L5 [path=LRLR; heap=20; reachable]
          f_1_5(x) = 0.26497515 * d2_b2_mmdt + 0.322656527 * n2_b2_mmdt - 0.656358636

      N10 [path=LRR; heap=10; UNREACHABLE (constant ancestor)]
        h_1_10(x) = 3.11856008e-17 * zlogz + 1.03393415e-17 * d2_b1_mmdt + 3.62950285e-17 *
↪ d2_a1_b1_mmdt - 3.214346e-17 * m2_b1_mmdt - 5.71317478e-17 * m2_b2_mmdt + 5.83355278e-17 * mass_mmdt +
↪  1.70316265e-11 * multiplicity - 1.89719708e-11
        <= 0 -> L6; > 0 -> L7

        L6 [path=LRRL; heap=21; UNREACHABLE (constant ancestor)]
          f_1_6(x) = 0

        L7 [path=LRRR; heap=22; UNREACHABLE (constant ancestor)]
          f_1_7(x) = 0

  N2 [path=R; heap=2; reachable]
    h_1_2(x) = 1.08273826 * c1_b2_mmdt + 0.24935691 * c2_b2_mmdt - 0.0971598576 * d2_b1_mmdt +
↪ 0.256354852 * d2_b2_mmdt - 0.0929252735 * d2_a1_b1_mmdt - 0.228292085 * m2_b1_mmdt + 0.0567631958 *
↪ mass_mmdt + 0.28027011 * multiplicity - 0.13138567
    <= 0 -> N5; > 0 -> N6
```

```
    N5 [path=RL; heap=5; reachable]
      h_1_5(x) = 0
      <= 0 -> N11; > 0 -> N12; CONSTANT: always LEFT
      N11 [path=RLL; heap=11; reachable]
        h_1_11(x) = -0.259890766 * d2_a1_b2_mmdt + 0.208572797 * m2_b1_mmdt + 3.31046124 * mass_mmdt -
↪ 0.0387863599
        <= 0 -> L8; > 0 -> L9
        L8 [path=RLLL; heap=23; reachable]
          f_1_8(x) = 1.49591861 * zlogz + 0.441575961 * c1_b0_mmdt + 1.71623752 * c1_b2_mmdt +
↪ 0.707286758 * d2_a1_b2_mmdt + 0.136799401 * n2_b1_mmdt - 0.0922364294 * n2_b2_mmdt - 4.1902612 *
↪ mass_mmdt + 0.994771372 * multiplicity + 0.902352018
        L9 [path=RLLR; heap=24; reachable]
          f_1_9(x) = 1.06616852 * zlogz + 0.26139261 * c1_b0_mmdt - 1.59073627 * c1_b1_mmdt - 3.31259981
↪  * c2_b1_mmdt + 2.90835606 * c2_b2_mmdt + 0.0402529973 * d2_b1_mmdt + 0.0402529973 * d2_a1_b1_mmdt +
↪ 3.30625 * d2_a1_b2_mmdt - 2.78491928 * m2_b1_mmdt + 0.342727361 * m2_b2_mmdt - 0.69660259 *
↪ n2_b2_mmdt + 1.00497391 * mass_mmdt + 2.54220983 * multiplicity + 0.0829818883
      N12 [path=RLR; heap=12; UNREACHABLE (constant ancestor)]
        h_1_12(x) = 3.32650029e-05 * c1_b2_mmdt + 1.11948505e-05 * c2_b2_mmdt + 2.49204468e-05 *
↪ d2_a1_b2_mmdt + 1.99509567e-06 * m2_b2_mmdt + 0.000137262248 * n2_b1_mmdt + 9.50083532e-05
        <= 0 -> L10; > 0 -> L11
        L10 [path=RLRL; heap=25; UNREACHABLE (constant ancestor)]
          f_1_10(x) = 0
        L11 [path=RLRR; heap=26; UNREACHABLE (constant ancestor)]
          f_1_11(x) = 0
    N6 [path=RR; heap=6; reachable]
      h_1_6(x) = 4.94065646e-324
      <= 0 -> N13; > 0 -> N14; CONSTANT: always RIGHT
      N13 [path=RRL; heap=13; UNREACHABLE (constant ancestor)]
        h_1_13(x) = -2.68948213e-10 * zlogz - 5.82051432e-11 * c2_b1_mmdt - 2.85373406e-10 * d2_b1_mmdt +
↪  2.05721037e-10 * d2_b2_mmdt - 1.06470833e-10 * d2_a1_b1_mmdt + 9.05405039e-12 * d2_a1_b2_mmdt +
↪ 3.42554664e-10 * m2_b2_mmdt + 2.13357549e-10 * n2_b2_mmdt - 8.89638165e-10 * mass_mmdt + 1.4924407e
↪ -09 * multiplicity - 1.72964941e-09
        <= 0 -> L12; > 0 -> L13
        L12 [path=RRLL; heap=27; UNREACHABLE (constant ancestor)]
          f_1_12(x) = 0
        L13 [path=RRLR; heap=28; UNREACHABLE (constant ancestor)]
          f_1_13(x) = 0
      N14 [path=RRR; heap=14; reachable]
        h_1_14(x) = -0.0357752902 * c1_b2_mmdt + 1.00738533 * mass_mmdt - 1.85526559
        <= 0 -> L14; > 0 -> L15
        L14 [path=RRRL; heap=29; reachable]
          f_1_14(x) = 0.992368325 * zlogz - 4.80052499 * c1_b1_mmdt + 2.90036981 * c1_b2_mmdt -
↪ 0.714839331 * c2_b1_mmdt - 0.349819173 * d2_b1_mmdt - 0.0440891415 * d2_b2_mmdt - 0.349819173 *
↪ d2_a1_b1_mmdt + 0.777214702 * d2_a1_b2_mmdt + 0.176693713 * m2_b1_mmdt + 2.01859804 * n2_b1_mmdt -
↪ 1.16458901 * n2_b2_mmdt - 1.53336443 * mass_mmdt + 0.283339646 * multiplicity + 2.63807278
        L15 [path=RRRR; heap=30; reachable]
          f_1_15(x) = -0.485616316 * c1_b0_mmdt - 0.625011056 * c2_b2_mmdt + 0.51939675 * d2_b1_mmdt +
↪ 0.199901858 * d2_b2_mmdt + 0.51939675 * d2_a1_b1_mmdt - 0.491209214 * m2_b2_mmdt + 2.38441955 *
↪ mass_mmdt - 0.280424752 * multiplicity - 5.0660948
Selected leaf f is a class-0 logit contribution; forest signal margin = -sum(f).
```

Implementation and reproducibility

Tree 2 (scientific tree $t = 3$).

```
TREE 2: depth=4, leaves=linear, input=standardized
Coefficient display: 9 significant digits; exact zeros omitted.
At every cut: h > 0 goes RIGHT; h <= 0 goes LEFT (including equality).
N0 [path=root; heap=0; reachable]
  h_2_0(x) = 0.451963069 * c1_b2_mmdt - 0.00922949319 * m2_b1_mmdt + 0.315856319 * mass_mmdt +
↪ 0.0990362496
  <= 0 -> N1; > 0 -> N2
  N1 [path=L; heap=1; reachable]
    h_2_1(x) = -0.0393006101 * zlogz + 0.0190956747 * multiplicity + 0.00401865025
    <= 0 -> N3; > 0 -> N4
```

```
    N3 [path=LL; heap=3; reachable]
      h_2_3(x) = 0
      <= 0 -> N7; > 0 -> N8; CONSTANT: always LEFT
      N7 [path=LLL; heap=7; reachable]
        h_2_7(x) = -0.338370941 * zlogz - 0.197456658 * c1_b0_mmdt + 0.505337081 * multiplicity +
↪ 0.323631119
        <= 0 -> L0; > 0 -> L1
        L0 [path=LLLL; heap=15; reachable]
          f_2_0(x) = 0.391007555 * c1_b0_mmdt + 2.7854501 * c1_b1_mmdt - 5.98969282 * c1_b2_mmdt +
↪ 0.733907917 * c2_b1_mmdt - 0.0568167023 * d2_b2_mmdt - 0.461051796 * m2_b1_mmdt + 0.358625148 *
↪ m2_b2_mmdt + 0.00378192334 * n2_b2_mmdt - 2.04581277 * mass_mmdt - 0.432532063 * multiplicity -
↪ 1.93961875
        L1 [path=LLLR; heap=16; reachable]
          f_2_1(x) = -0.441179138 * zlogz - 3.22570484 * c1_b2_mmdt - 0.0681594455 * d2_b1_mmdt -
↪ 0.0681594455 * d2_a1_b1_mmdt + 0.311882288 * d2_a1_b2_mmdt - 0.131522611 * n2_b1_mmdt - 0.234805732 *
↪  mass_mmdt + 0.648265323 * multiplicity - 0.763080965
      N8 [path=LLR; heap=8; UNREACHABLE (constant ancestor)]
        h_2_8(x) = 0.00394358152 * mass_mmdt - 0.000176803622
        <= 0 -> L2; > 0 -> L3
        L2 [path=LLRL; heap=17; UNREACHABLE (constant ancestor)]
          f_2_2(x) = 0
        L3 [path=LLRR; heap=18; UNREACHABLE (constant ancestor)]
          f_2_3(x) = 0
    N4 [path=LR; heap=4; reachable]
      h_2_4(x) = 0.748689614 * c1_b0_mmdt + 0.669457034 * c2_b1_mmdt + 0.101051343
      <= 0 -> N9; > 0 -> N10
      N9 [path=LRL; heap=9; reachable]
        h_2_9(x) = 0
        <= 0 -> L4; > 0 -> L5; CONSTANT: always LEFT
        L4 [path=LRLL; heap=19; reachable]
          f_2_4(x) = -0.389784207 * zlogz - 0.209597106 * c1_b0_mmdt - 3.08379498 * c1_b1_mmdt -
↪ 1.4593788 * c2_b1_mmdt + 0.0255711958 * d2_b1_mmdt - 0.0574990566 * d2_b2_mmdt + 0.0255711958 *
↪ d2_a1_b1_mmdt + 0.973389446 * d2_a1_b2_mmdt + 0.250244728 * m2_b2_mmdt - 0.782019914 * n2_b1_mmdt +
↪ 0.0194565833 * n2_b2_mmdt + 2.61367386 * mass_mmdt + 0.993769728 * multiplicity + 0.228651881
        L5 [path=LRLR; heap=20; UNREACHABLE (constant ancestor)]
          f_2_5(x) = 0
      N10 [path=LRR; heap=10; reachable]
        h_2_10(x) = -0.615381338 * zlogz - 0.780626703 * multiplicity - 0.248174582
        <= 0 -> L6; > 0 -> L7
        L6 [path=LRRL; heap=21; reachable]
          f_2_6(x) = 0.405855831 * zlogz + 1.19454522 * c1_b0_mmdt - 0.171973875 * c1_b1_mmdt -
↪ 3.01430074 * c2_b1_mmdt + 1.66035781 * c2_b2_mmdt + 0.195722301 * d2_b2_mmdt - 0.470821689 *
↪ d2_a1_b2_mmdt + 0.725279387 * m2_b1_mmdt - 0.0949253938 * m2_b2_mmdt + 1.53492 * n2_b1_mmdt -
↪ 0.872843694 * n2_b2_mmdt + 1.27018591 * mass_mmdt + 0.644659641 * multiplicity + 0.542419575
        L7 [path=LRRR; heap=22; reachable]
          f_2_7(x) = 3.48806978 * zlogz + 7.81025712 * c1_b0_mmdt - 0.95346229 * c1_b2_mmdt +
↪ 0.280675132 * c2_b1_mmdt + 0.0334405957 * d2_b2_mmdt - 1.01384781 * d2_a1_b2_mmdt + 1.85257453 *
↪ m2_b1_mmdt - 0.533779879 * m2_b2_mmdt - 0.647558288 * n2_b1_mmdt + 0.591781799 * n2_b2_mmdt -
↪ 1.08156364 * mass_mmdt + 1.20551885 * multiplicity - 2.42799078
  N2 [path=R; heap=2; reachable]
    h_2_2(x) = 0.168562252 * c2_b2_mmdt - 0.0982298021 * d2_a1_b2_mmdt + 0.37090396 * m2_b1_mmdt +
↪ 0.113712418 * multiplicity - 0.745275257
    <= 0 -> N5; > 0 -> N6
    N5 [path=RL; heap=5; reachable]
      h_2_5(x) = 0.814138721 * mass_mmdt + 0.0531775224 * multiplicity - 0.29709854
      <= 0 -> N11; > 0 -> N12
      N11 [path=RLL; heap=11; reachable]
        h_2_11(x) = 4.94065646e-324
        <= 0 -> L8; > 0 -> L9; CONSTANT: always RIGHT
        L8 [path=RLLL; heap=23; UNREACHABLE (constant ancestor)]
          f_2_8(x) = 0
        L9 [path=RLLR; heap=24; reachable]
          f_2_9(x) = -0.273060581 * zlogz + 0.214560559 * c1_b0_mmdt + 2.79082823 * c1_b2_mmdt +
↪ 0.0872642558 * d2_b1_mmdt + 0.0872642558 * d2_a1_b1_mmdt + 0.0668304306 * d2_a1_b2_mmdt + 0.700471572
↪  * m2_b2_mmdt - 0.377933505 * n2_b2_mmdt + 0.446788216 * multiplicity + 0.784109979
```

```
      N12 [path=RLR; heap=12; reachable]
        h_2_12(x) = 2.06493224e-10 * c1_b2_mmdt + 6.8828261e-11 * c2_b1_mmdt + 2.89598635e-11 *
↪ m2_b1_mmdt + 5.35347149e-11 * m2_b2_mmdt - 1.75510092e-10
        <= 0 -> L10; > 0 -> L11
        L10 [path=RLRL; heap=25; reachable]
          f_2_10(x) = 0.0172917486 * zlogz + 0.479280898 * c1_b0_mmdt + 0.895727863 * c1_b1_mmdt -
↪ 2.47138391 * c1_b2_mmdt + 1.46797745 * c2_b1_mmdt + 0.0674431371 * d2_b1_mmdt - 0.20536907 *
↪ d2_b2_mmdt + 0.0674431371 * d2_a1_b1_mmdt + 0.960087986 * m2_b2_mmdt - 0.399753552 * n2_b1_mmdt -
↪ 0.688302486 * n2_b2_mmdt + 5.79450008 * mass_mmdt + 0.0327603624 * multiplicity - 0.350562803
        L11 [path=RLRR; heap=26; reachable]
          f_2_11(x) = 0.851042201 * c1_b0_mmdt + 1.12087708 * c1_b2_mmdt - 2.14848846 * c2_b1_mmdt +
↪ 0.740207807 * d2_b1_mmdt + 0.740207807 * d2_a1_b1_mmdt + 1.30902686 * m2_b1_mmdt + 4.77523926 *
↪ mass_mmdt + 1.62965024
    N6 [path=RR; heap=6; reachable]
      h_2_6(x) = 0
      <= 0 -> N13; > 0 -> N14; CONSTANT: always LEFT
      N13 [path=RRL; heap=13; reachable]
        h_2_13(x) = 4.94065646e-324
        <= 0 -> L12; > 0 -> L13; CONSTANT: always RIGHT
        L12 [path=RRLL; heap=27; UNREACHABLE (constant ancestor)]
          f_2_12(x) = 0
        L13 [path=RRLR; heap=28; reachable]
          f_2_13(x) = -0.148614563 * zlogz + 1.11851788 * c1_b0_mmdt + 0.96743728 * c1_b1_mmdt +
↪ 0.00568823559 * c2_b2_mmdt + 0.40419135 * d2_b2_mmdt - 1.01497415 * d2_a1_b2_mmdt + 1.11598924 *
↪ m2_b1_mmdt + 1.21950507 * n2_b1_mmdt - 0.144073893 * n2_b2_mmdt + 4.08231683 * mass_mmdt -
↪ 0.113704284 * multiplicity - 0.734827284
      N14 [path=RRR; heap=14; UNREACHABLE (constant ancestor)]
        h_2_14(x) = 0.078412842 * c1_b2_mmdt - 0.128921279
        <= 0 -> L14; > 0 -> L15
        L14 [path=RRRL; heap=29; UNREACHABLE (constant ancestor)]
          f_2_14(x) = 0
        L15 [path=RRRR; heap=30; UNREACHABLE (constant ancestor)]
          f_2_15(x) = 0
Selected leaf f is a class-0 logit contribution; forest signal margin = -sum(f).
```